

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

“JnanaSangama”, Belgaum -590014, Karnataka.



LAB REPORT

On

ANALYSIS AND DESIGN OF ALGORITHMS (23CS4PCADA)

Submitted by

Harsh Chandra Saini(1WA24CS122)

**in partial fulfillment for the award of the degree of
BACHELOR OF ENGINEERING
in
COMPUTER SCIENCE AND ENGINEERING**



**B.M.S. COLLEGE OF ENGINEERING
(Autonomous Institution under VTU)
BENGALURU-560019
February to June 2026**

CERTIFICATE

**B. M. S. College of Engineering,
Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering**



This is to certify that the Lab work entitled “**ANALYSIS AND DESIGN OF ALGORITHMS**” carried out by **Harsh Chandra Saini (1WA24CS122)**, who is bonafide student of **B. M. S. College of Engineering**. It is in partial fulfillment for the award of **Bachelor of Engineering in Computer Science and Engineering** of the Visvesvaraya Technological University, Belgaum during the year 2025-26. The Lab report has been approved as it satisfies the academic requirements in respect of Analysis and Design of Algorithms Lab - **(23CS4PCADA)** work prescribed for the said degree.

Manjula S
Assistant Professor
Department of CSE
BMSCE, Bengaluru

Dr. Kavitha Sooda
Professor and Head
Department of CSE
BMSCE, Bengaluru

Index Sheet

Sl. No.	Experiment Title	Page No.
1	Write program to obtain the Topological ordering of vertices in a given digraph.	4-6
2	Implement Johnson Trotter algorithm to generate permutations.	7-8
3	Sort a given set of N integer elements using Merge Sort technique and compute its time taken. Run the program for different values of N and record the time taken to sort.	9-12
4	Sort a given set of N integer elements using Quick Sort technique and compute its time taken.	13-16
5	Sort a given set of N integer elements using Heap Sort technique and compute its time taken.	17-19
6	Implement 0/1 Knapsack problem using dynamic programming. (5 Marks)	20-21
7	Implement All Pair Shortest paths problem using Floyd's algorithm.	22-23
8	(a) Find Minimum Cost Spanning Tree of a given undirected graph using Prim's algorithm. (b) Find Minimum Cost Spanning Tree of a given undirected graph using Kruskal's algorithm.	24-27
9	Implement Fractional Knapsack using Greedy technique.	28-30
10	From a given vertex in a weighted connected graph, find shortest paths to other vertices using Dijkstra's algorithm.	31-32
11	Implement "N-Queens Problem" using Backtracking.	33-34

Course outcomes:

CO1	Analyze time complexity of algorithms using asymptotic notations.
CO2	Apply various algorithm design techniques for the given problem.
CO3	Evaluate the appropriate algorithm/design technique for solving a given problem.
CO4	Conduct practical experiments by applying appropriate algorithm design technique to solve problems.

Lab program 1:

Write program to obtain the Topological ordering of vertices in a given digraph.

SOURCE CODE

```
#include <stdio.h>
#include <stdlib.h>
#define MAX 100

int adj[MAX][MAX];
int indegree[MAX];
int queue[MAX];
int n;

void topologicalSort() {
    int front = 0, rear = -1;
    int count = 0;
    for (int i = 0; i < n; i++)
        indegree[i] = 0;

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            if (adj[i][j] == 1)
                indegree[j]++;
        }
    }

    for (int i = 0; i < n; i++) {
        if (indegree[i] == 0)
            queue[++rear] = i;
    }

    printf("\nTopological Order: ");

    while (front <= rear) {
        int node = queue[front++];
        printf("%d ", node);
        count++;
        for (int i = 0; i < n; i++) {
            if (adj[node][i] == 1) {
                indegree[i]--;
                if (indegree[i] == 0)
                    queue[++rear] = i;
            }
        }
    }

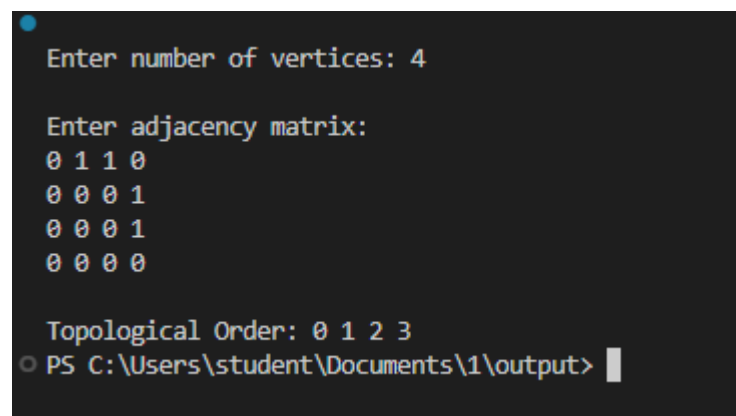
    if (count != n)
        printf("\nGraph has a cycle! No topological ordering possible.\n");
}
```

```
int main() {
    printf("\nEnter number of vertices: ");
    scanf("%d", &n);

    printf("\nEnter adjacency matrix:\n");
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++)
            scanf("%d", &adj[i][j]);
    }

    topologicalSort();
    return 0;
}
```

OUTPUT



The screenshot shows a terminal window with a dark background. The user enters '4' for the number of vertices. Then, they enter a 4x4 adjacency matrix row by row: '0 1 1 0', '0 0 0 1', '0 0 0 1', and '0 0 0 0'. The program outputs 'Topological Order: 0 1 2 3'. The prompt 'PS C:\Users\student\Documents\1\output>' is visible at the bottom.

```
Enter number of vertices: 4

Enter adjacency matrix:
0 1 1 0
0 0 0 1
0 0 0 1
0 0 0 0

Topological Order: 0 1 2 3
PS C:\Users\student\Documents\1\output>
```

Leetcode 207: Course Schedule (Topological Sort)

Problem List

207. Course Schedule

Solved

Medium

Topics

Companies

Hint

There are a total of `numCourses` courses you have to take, labeled from `0` to `numCourses - 1`. You are given an array `prerequisites` where `prerequisites[i] = [ai, bi]` indicates that you **must** take course `bi` first if you want to take course `ai`.

- For example, the pair `[0, 1]`, indicates that to take course `0` you have to first take course `1`.

Return `true` if you can finish all courses. Otherwise, return `false`.

Example 1:

Input: `numCourses = 2, prerequisites = [[1,0]]`
Output: `true`
Explanation: There are a total of 2 courses to take. To take course 1 you should have finished course 0. So it is possible.

Example 2:

Input: `numCourses = 2, prerequisites = [[1,0],[0,1]]`
Output: `false`
Explanation: There are a total of 2 courses to take. To take course 1 you should have finished course 0, and to take course 0 you should also have finished course 1. So it is impossible.

Constraints:

- `1 <= numCourses <= 2000`
- `0 <= prerequisites.length <= 5000`

Code

C Auto

```
1 bool canFinish(int numCourses, int** prerequisites, int prerequisitesSize,
2 int* prerequisitesColSize) {
3     // Step 1: Build indegree array and adjacency list
4     int* indegree = (int*)calloc(numCourses, sizeof(int));
5     int** adj = (int**)malloc(sizeof(int*) * numCourses);
6     int* adjSize = (int*)calloc(numCourses, sizeof(int));
7
8     for (int i = 0; i < numCourses; i++) {
9         adj[i] = (int*)malloc(sizeof(int) * numCourses); // worst case
10    }
11
12    for (int i = 0; i < prerequisitesSize; i++) {
13        int course = prerequisites[i][0];
14        int prereq = prerequisites[i][1];
15        adj[prereq][adjSize[prereq]++] = course;
16        indegree[course]++;
17    }
18}
```

Saved Ln 44, Col 2

Testcase

Test Result

Accepted Runtime: 0 ms

Case 1

Case 2

Input

`numCourses =`
`2`

`prerequisites =`
`[[1,0]]`

Output

`true`

Runtime

33 ms | Beats 46.82%

Memory

109.74 MB | Beats 48.84%

Runtime (ms)	Percentage
0-5	12%
5-10	4%
10-15	3%
15-20	2%
20-25	1%
25-30	2%
30-35	3%
35-40	4%
40-45	5%
45-50	2%
50-55	1%
55-60	1%

6 | Page

Lab program 2:

Implement Johnson Trotter algorithm to generate permutations.

SOURCE CODE

```
#include <stdio.h>
int a[10], dir[10];

void print(int n){
    for (int i=0;i<n; i++)
        printf("%d ", a[i]);
    printf("\n");
}

int mobile(int n){
    int m=0,pos=-1;

    for (int i=0; i<n; i++){
        int j=i+dir[i];
        if (j >= 0 && j < n && a[i] > a[j] && a[i] > m) {
            m=a[i];
            pos=i;
        }
    }
    return pos;
}

int main(){
    int n;
    printf("\nEnter number: ");
    scanf("%d", &n);

    for (int i=0;i<n;i++) {
        a[i]=i+1;
        dir[i] = -1;
    }

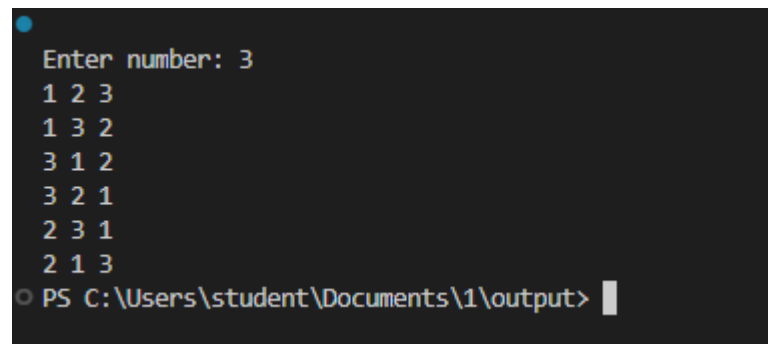
    print(n);

    while (1) {
        int i = mobile(n);
        if (i == -1) break;
        int j = i + dir[i];
        int t = a[i]; a[i] = a[j]; a[j] = t;
        t = dir[i]; dir[i] = dir[j]; dir[j] = t;

        for (int k = 0; k < n; k++)
            if (a[k] > a[j]) dir[k] *= -1;
    }
}
```

```
    print(n);  
}  
  
return 0;  
}
```

OUTPUT



```
Enter number: 3  
1 2 3  
1 3 2  
3 1 2  
3 2 1  
2 3 1  
2 1 3  
2 1 3  
PS C:\Users\student\Documents\1\output>
```

Lab program 3:

Sort a given set of N integer elements using Merge Sort technique and compute its time taken. Run the program for different values of N and record the time taken to sort.

SOURCE CODE

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int a[5000000],b[5000000];

void merge(int a[],int low,int mid,int high){

    int i=low,j=mid+1,k=low;

    while(i<=mid && j<=high){
        if(a[i]<a[j])
            b[k]=a[i++];
        else
            b[k]=a[j++];
        k++;
    }
    while(i<=mid){
        b[k++]=a[i++];
    }
    while(j<=high){
        b[k++]=a[j++];
    }
    for(int x=low;x<=high;x++)
        a[x]=b[x];
}

void merge_sort(int a[],int low,int high){
    if(low<high){
        int mid=(low+high)/2;
        merge_sort(a,low,mid);
        merge_sort(a,mid+1,high);
        merge(a,low,mid,high);
    }
}
```

```

int main(){
    int n;
    printf("\nEnter number of elements: ");
    scanf("%d",&n);
    printf("\nEnter the elements...");
    for(int i=0;i<n;i++){
        a[i]=rand()%1000;
        printf("%d ",a[i]);
    }
    clock_t start_time=clock();
    merge_sort(a,0,n-1);
    clock_t end_time=clock();
    printf("\nAfter sorting: ");
    for(int i=0;i<n;i++)
        printf("%d ",a[i]);
    float total_time=(float)(end_time-start_time)/CLOCKS_PER_SEC*1000;
    printf("\nTime taken is: %.2f",total_time);
    return 0;
}

```

OUTPUT

```

Enter number of elements: 10

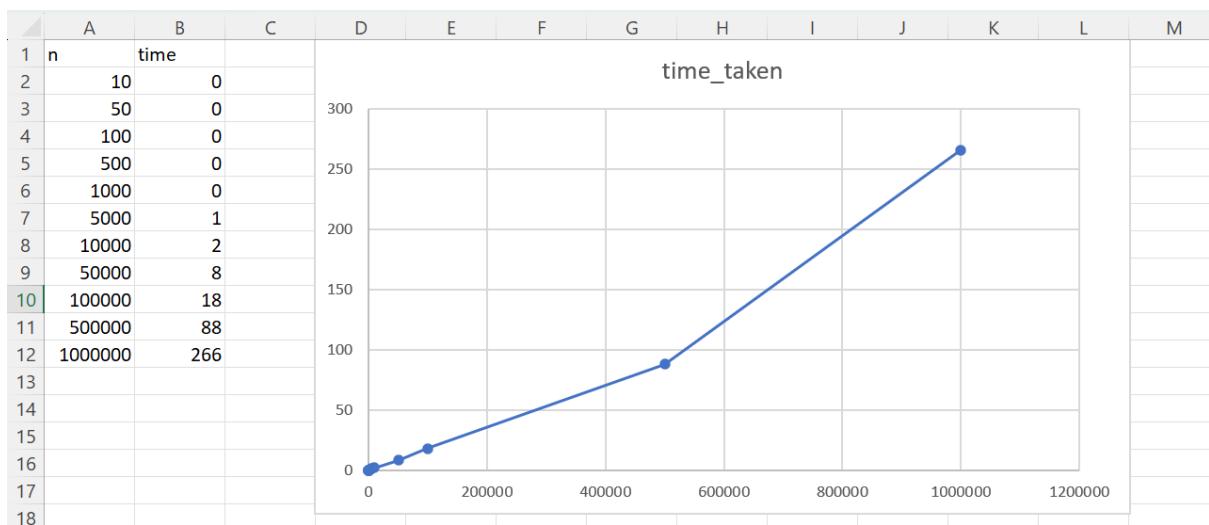
Enter the elements...41 467 334 500 169 724 478 358 962 464
After sorting: 41 169 334 358 464 467 478 500 724 962
Time taken is: 0.00
PS C:\Users\student\Documents\1\output>

```

Time taken for each N

N	Time taken
10	0.0
50	0.0
100	0.0
500	0.0
1000	0.0
5000	1.0
10000	2.0
50000	8.0
100000	18.0
500000	88.0
1000000	266.0

Graph:



Leetcode 148: Sort List

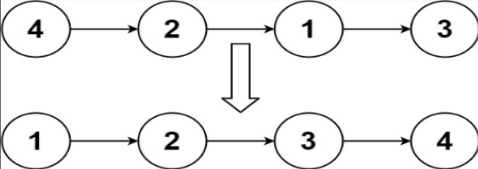
Description

148. Sort List

Medium Topics Companies

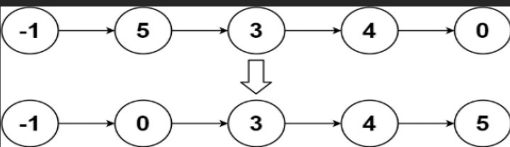
Given the `head` of a linked list, return the list after sorting it in **ascending order**.

Example 1:



Input: head = [4,2,1,3]
Output: [1,2,3,4]

Example 2:



Input: head = [-1,5,3,4,0]
Output: [-1,0,3,4,5]

Code

```
32 return head;
33
34
35 struct ListNode* slow = head, *fast = head, *prev = NULL;
36 while (fast && fast->next) {
37     prev = slow;
38     slow = slow->next;
39     fast = fast->next->next;
40 }
41 prev->next = NULL;
42
43 struct ListNode* left = sortList(head);
44 struct ListNode* right = sortList(slow);
45
46 return merge(left, right);
47 }
```

Saved Ln 47, Col 2

Testcase

Test Result

Accepted Runtime: 0 ms

Case 1 Case 2 Case 3

Input

head = [4,2,1,3]

Output

[1,2,3,4]

Expected

[1,2,3,4]

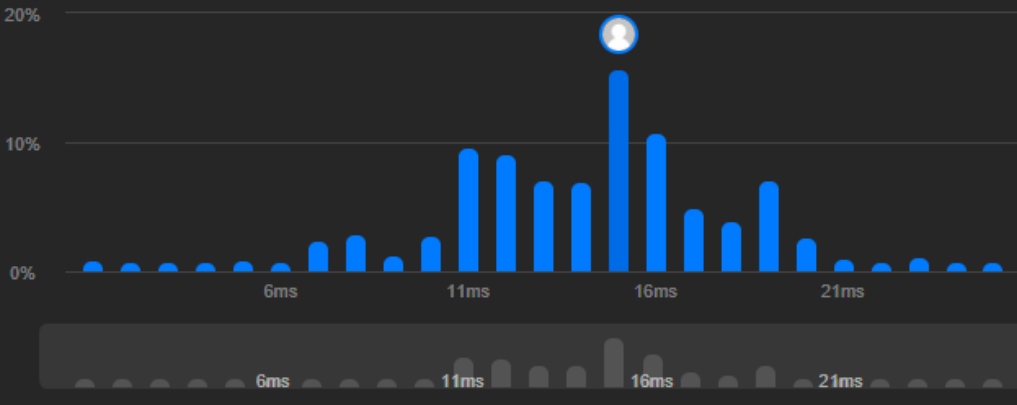
13.1K 191 114 Online

Runtime

15 ms | Beats 54.10%

Memory

25.62 MB | Beats 16.92%



Code | C

12 | Page

Lab program 4:

Sort a given set of N integer elements using Quick Sort technique and compute its time taken.

SOURCE CODE

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
int a[5000000];

void swap(int *a,int *b){
    int temp=*a;
    *a=*b;
    *b=temp;
}

int partition(int a[],int low,int high){

    int pivot=a[low];
    int i=low+1;
    int j=high;

    while(i<=j){
        while(i<=high && pivot>=a[i])
            i++;
        while(j>=low && pivot<a[j])
            j--;
        if(i<j)
            swap(&a[i],&a[j]);
    }
    swap(&a[low],&a[j]);
    return j;
}

void quick_sort(int a[],int low,int high){
    if(low<high){
        int j=partition(a,low,high);
        quick_sort(a,low,j-1);
        quick_sort(a,j+1,high);
    }
}
```

```

int main(){

    int n;
    printf("\nEnter number of elements: ");
    scanf("%d",&n);

    printf("\nEnter elements...");
    for(int i=0;i<n;i++){
        a[i]=rand()%1000;
        printf("%d ",a[i]);
    }

    clock_t start=clock();
    quick_sort(a,0,n-1);
    clock_t end=clock();

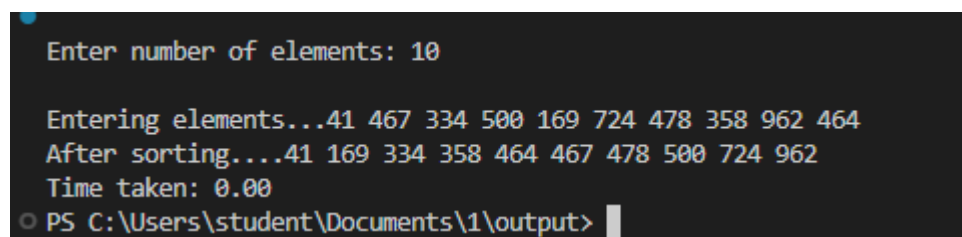
    printf("\nAfter sorting....");
    for(int i=0;i<n;i++)
        printf("%d ",a[i]);

    float time_taken=((float)(end-start))/CLOCKS_PER_SEC*1000;
    printf("\nTime taken: %.2f",time_taken);

    return 0;
}

```

OUTPUT



```

Enter number of elements: 10

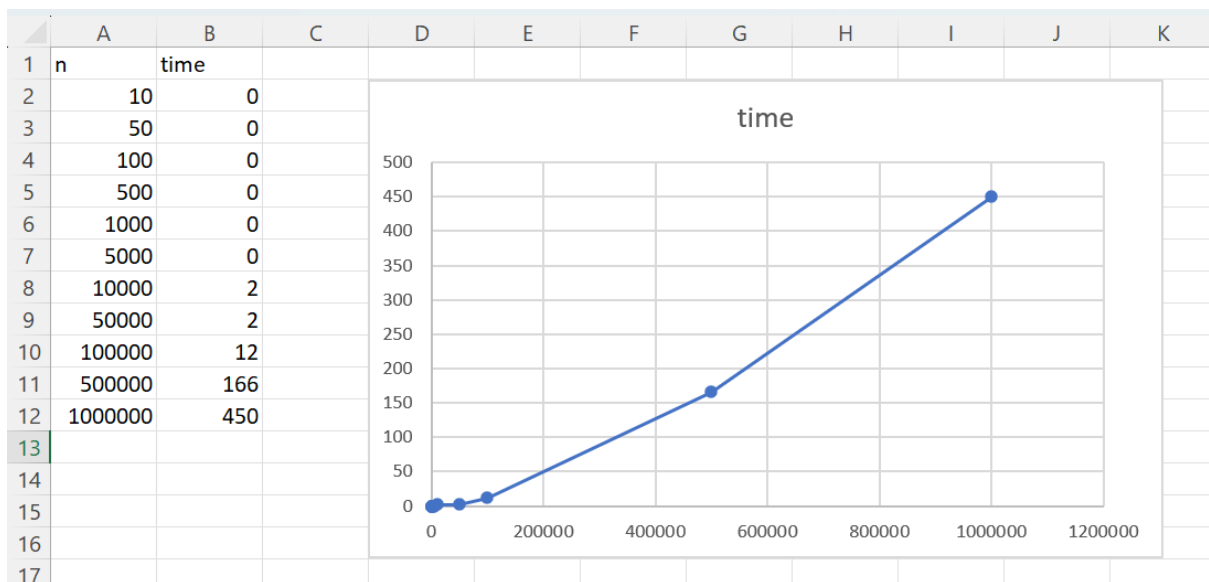
Entering elements...41 467 334 500 169 724 478 358 962 464
After sorting....41 169 334 358 464 467 478 500 724 962
Time taken: 0.00
PS C:\Users\student\Documents\1\output>

```

Time taken for each N

N	Time taken
10	0.0
50	0.0
100	0.0
500	0.0
1000	0.0
5000	0.0
10000	2.0
50000	2.0
100000	12.0
500000	166.0
1000000	450.0

Graph:



Leetcode 215: kth Largest Element in an Array

Description

215. Kth Largest Element in an Array

Medium

Topics

Companies

Given an integer array `nums` and an integer `k`, return the k^{th} largest element in the array.

Note that it is the k^{th} largest element in the sorted order, not the k^{th} distinct element.

Can you solve it without sorting?

Example 1:

Input: `nums = [3,2,1,5,6,4]`, `k = 2`
Output: 5

Example 2:

Input: `nums = [3,2,3,1,2,4,5,5,6]`, `k = 4`
Output: 4

Constraints:

- $1 \leq k \leq \text{nums.length} \leq 10^5$
- $-10^4 \leq \text{nums}[i] \leq 10^4$

Seen this question in a real interview before? 1/6

Code

```
44
45     if (smallest != index) {
46         swap(&heap->arr[index], &heap->arr[smallest]);
47         heapifyDown(heap, smallest);
48     }
49 }
50
51 void insert(struct MinHeap* heap, int value) {
52     if (heap->size == heap->capacity) {
53         printf("Heap is full\n");
54         return;
55     }
56
57     heap->arr[heap->size] = value;
58     heap->size++;
59     heapifyUp(heap, heap->size - 1);
60 }
61
```

SavedLn 80, Col 2

Testcase

Test Result

AcceptedRuntime: 0 ms

Case 1

Case 2

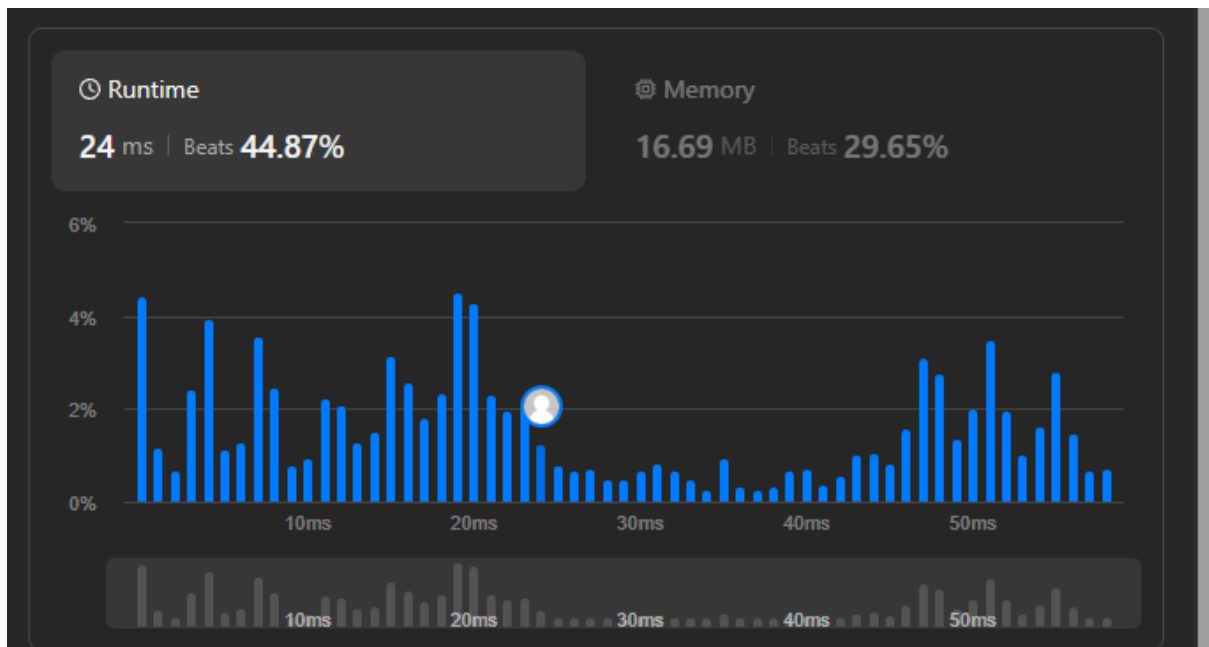
Input

nums =
[3,2,1,5,6,4]

k =
2

Output

5



Lab program 5:

Sort a given set of N integer elements using Heap Sort technique and compute its time taken.

SOURCE CODE

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int a[5000000];

void heapify(int a[],int i,int n){
    int largest=i;
    int left=2*i+1;
    int right=2*i+2;

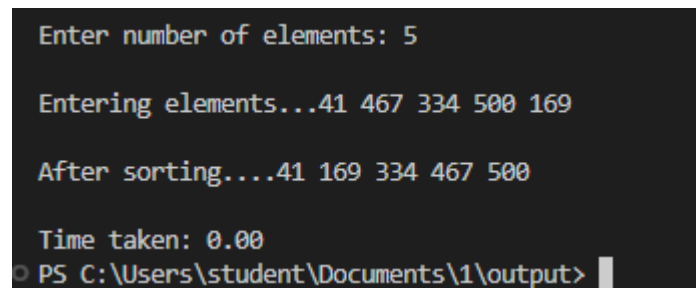
    if(left<n && a[left]>a[largest])
        largest=left;
    if(right<n && a[right]>a[largest])
        largest=right;
    if(largest!=i){
        int temp=a[i];
        a[i]=a[largest];
        a[largest]=temp;
        heapify(a,largest,n);
    }
}

void heap_sort(int a[],int n){
    for(int i=(n/2)-1;i>=0;i--){
        heapify(a,i,n);
    }
    for(int i=n-1;i>0;i--){
        int temp=a[0];
        a[0]=a[i];
        a[i]=temp;
        heapify(a,0,i);
    }
}

void print(int a[],int n){
    for(int i=0;i<n;i++){
        printf("%d ",a[i]);
        printf("\n");
    }
}
```

```
int main(){
    int n;
    printf("\nEnter number of elements: ");
    scanf("%d",&n);
    printf("\nEnter elements...");
    for(int i=0;i<n;i++){
        a[i]=rand()%1000;
    }
    print(a,n);
    clock_t start=clock();
    heap_sort(a,n);
    clock_t end=clock();
    printf("\nAfter sorting....");
    print(a,n);
    float time_taken=((float)(end-start))/CLOCKS_PER_SEC*1000;
    printf("\nTime taken: %.2f",time_taken);
    return 0;
}
```

OUTPUT



The screenshot shows a terminal window with a dark background. The text is as follows:

```
Enter number of elements: 5

Entering elements...41 467 334 500 169

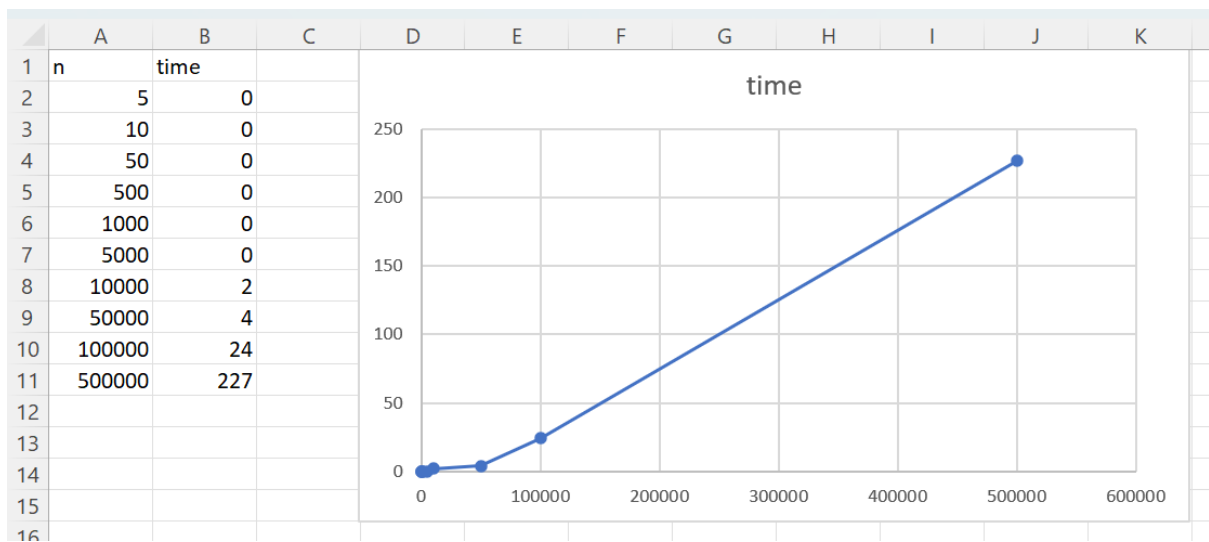
After sorting....41 169 334 467 500

Time taken: 0.00
PS C:\Users\student\Documents\1\output>
```

Time taken for each N

N	Time taken
5	0
10	0
50	0
500	0
1000	0
5000	0
10000	2
50000	4
100000	24
500000	227
5	0

Graph:



Lab program 6:

Implement 0/1 Knapsack problem using dynamic programming.

SOURCE CODE

```
#include <stdio.h>

int p[50],w[50],n,m;
void inputs(){
    printf("\nEnter number of items: ");
    scanf("%d",&n);
    printf("\nEnter capacity: ");
    scanf("%d",&m);

    printf("\nEnter weights: ");
    for(int i=1;i<=n;i++)
        scanf("%d",&w[i]);
    printf("\nEnter profits: ");
    for(int i=1;i<=n;i++)
        scanf("%d",&p[i]);
}

int max(int a,int b){
    return (a>b)?a:b;
}

int knapsack(){
    int v[n+1][m+1];
    for(int i=0;i<=n;i++){
        for(int j=0;j<=m;j++){
            if(i==0||j==0)
                v[i][j]=0;
            else if(w[i]>j)
                v[i][j]=v[i-1][j];
            else
                v[i][j]=max(v[i-1][j],(v[i-1][j-w[i]])+p[i]);
        }
    }
    printf("\nOptimal Profit: %d",v[n][m]);
}

int main(){
    inputs();
    knapsack();
    return 0;
}
```

OUTPUT

```

Enter number of items: 4

Enter capacity: 8

Enter weights: 3 4 6 5

Enter profits: 2 3 1 4

Optimal Profit: 6
PS C:\Users\student\Documents\1\output>

```

Leetcode 338: counting Bits

Problem List

338. Counting Bits

Easy Topics Companies Hint

Given an integer n , return an array `ans` of length $n + 1$ such that for each i ($0 \leq i \leq n$), `ans[i]` is the number of 1's in the binary representation of i .

Example 1:

Input: $n = 2$
Output: `[0, 1, 1]`
Explanation:
0 → 0
1 → 1
2 → 10

Example 2:

Input: $n = 5$
Output: `[0, 1, 1, 2, 1, 2]`
Explanation:
0 → 0
1 → 1
2 → 10
3 → 11
4 → 100
5 → 101

Constraints:

- $0 \leq n \leq 10^5$

Follow up:

12K 127 72 Online

Code

```

1 /**
2  * Note: The returned array must be malloced, assume caller calls free().
3  */
4 int* countBits(int n, int* returnSize){
5     int* ans = (int*)malloc((n+1)*sizeof(int));
6     *returnSize = n+1;
7     ans[0] = 0;
8     for(int i=1; i<=n; i++){
9         ans[i] = ans[i/2] + (i % 2);
10    }
11    return ans;
12 }

```

Testcase Test Result

Accepted Runtime: 0 ms

Case 1 Case 2

Input

$n =$

2

Output

`[0, 1, 1]`

Expected

`[0, 1, 1]`

Runtime

0 ms | Beats 100.00%

Memory

15.72 MB | Beats 34.58%

67.61% of solutions used 0 ms of runtime

1ms 2ms 3ms 4ms

Lab program 7:

Implement All Pair Shortest paths problem using Floyd's algorithm.

SOURCE CODE

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#define INF 99999

void floyd(int n,int R[n][n]){
    for(int k=0;k<n;k++){
        for(int i=0;i<n;i++){
            for(int j=0;j<n;j++){
                if(R[i][j]>(R[i][k]+R[k][j]))
                    R[i][j]=R[i][k]+R[k][j];
            }
        }
    }
}

int main(){
    int n;
    printf("\nEnter number of vertices: ");
    scanf("%d",&n);
    int R[n][n];
    printf("\nEnter adjacency matrix:\n");
    for(int i=0;i<n;i++){
        for(int j=0;j<n;j++){
            scanf("%d",&R[i][j]);
        }
    }
    floyd(n,R);
    printf("\nShortest path:\n");
    for(int i=0;i<n;i++){
        for(int j=0;j<n;j++){
            if(R[i][j]==INF)
                printf("INF ");
            else
                printf("%d ",R[i][j]);
        }
        printf("\n");
    }
    return 0;
}
```

OUTPUT

```
Enter number of vertices: 4

Enter adjacency matrix:
0 99999 3 99999
2 0 99999 99999
99999 7 0 1
6 99999 99999 0

Shortest path:
0 10 3 4
2 0 5 6
7 7 0 1
6 16 9 0

PS C:\Users\student\Documents\1\output>
```

Leetcode 1791: Center of Star Graph

1791. Find Center of Star Graph

Easy Topics Companies Hint

There is an undirected **star** graph consisting of n nodes labeled from 1 to n . A star graph is a graph where there is one **center** node and **exactly** $n - 1$ edges that connect the center node with every other node.

You are given a 2D integer array `edges` where each `edges[i] = [ui, vi]` indicates that there is an edge between the nodes u_i and v_i . Return the center of the given star graph.

Example 1:

```
graph TD; 2 --- 1; 2 --- 3; 2 --- 4;
```

Input: `edges = [[1,2],[2,3],[4,2]]`
Output: `2`
Explanation: As shown in the figure above, node 2 is connected to every other node, so 2 is the center.

Example 2:

```
1 int findCenter(int** edges, int edgesSize, int* edgesColSize) {
2
3     if(edges[0][0] == edges[1][0] || edges[0][0] == edges[1][1]){
4         return edges[0][0];
5     }
6     else if(edges[0][1] == edges[1][0] || edges[0][1] == edges[1][1]){
7         return edges[0][1];
8     }
9     else{
10        return 0;
11    }
12 }
```

Accepted Runtime: 0 ms

Case 1 Case 2

Input

`edges =`
`[[1,2],[2,3],[4,2]]`

Output

`2`

Expected

`2`

Runtime

0 ms | Beats 100.00%

Memory

34.62 MB | Beats 67.93%

89.87% of solutions used 0 ms of runtime

Runtime (ms)	Percentage of Solutions
0	89.87%
1	
3	
4	
7	

Lab program 8:

(a) Find Minimum Cost Spanning Tree of a given undirected graph using Prim's algorithm.

SOURCE CODE

```
#include <stdio.h>

void prims(int n,int graph[n][n]){

    int visited[n];
    for(int i=0;i<n;i++){
        visited[i]=0;
    }
    int edges=0;
    visited[0]=1;
    int cost=0;
    printf("\nEdge\tWeight\n");

    while(edges<n-1){

        int min=100,x=-1,y=-1;
        for(int i=0;i<n;i++){
            if(visited[i]==1){
                for(int j=0;j<n;j++){
                    if(visited[j]==0 && graph[i][j]!=0){
                        if(graph[i][j]<min){
                            min=graph[i][j];
                            x=i;
                            y=j;
                        }
                    }
                }
            }
        }

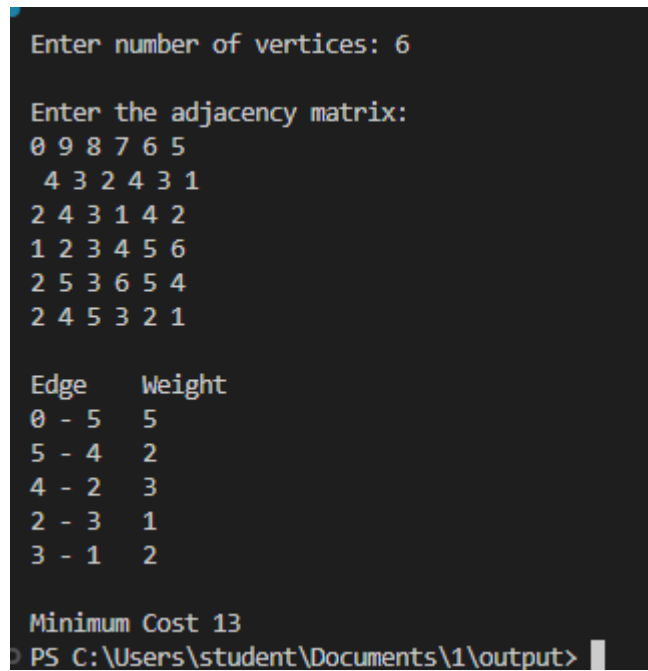
        printf("%d - %d\t%d\n",x,y,graph[x][y]);
        cost+=graph[x][y];
        visited[y]=1;
        edges++;
    }
    printf("\nMinimum Cost %d\n",cost);
}
```

```

int main(){
    int n;
    printf("\nEnter number of vertices: ");
    scanf("%d",&n);
    int graph[n][n];
    printf("\nEnter the adjacency matrix:\n");
    for(int i=0;i<n;i++){
        for(int j=0;j<n;j++){
            scanf("%d",&graph[i][j]);
        }
    }
    prims(n,graph);
    return 0;
}

```

OUTPUT



```

Enter number of vertices: 6

Enter the adjacency matrix:
0 9 8 7 6 5
4 3 2 4 3 1
2 4 3 1 4 2
1 2 3 4 5 6
2 5 3 6 5 4
2 4 5 3 2 1

Edge    Weight
0 - 5    5
5 - 4    2
4 - 2    3
2 - 3    1
3 - 1    2

Minimum Cost 13
PS C:\Users\student\Documents\1\output>

```

Lab program 8:

(b) Find Minimum Cost Spanning Tree of a given undirected graph using Kruskal's algorithm.

SOURCE CODE

```
#include <stdio.h>

struct Edge{
    int u,v,w;
}e[10],temp;

int parent[10];

int find(int x){
    while(parent[x]!=x)
        x=parent[x];
    return x;
}

int main(){

    int n,m,cost=0;

    printf("\nEnter number of Vertices and Edges: ");
    scanf("%d%d",&n,&m);

    printf("Enter vertex u v and weight w:\n");

    for(int i=0;i<m;i++){
        scanf("%d%d%d",&e[i].u,&e[i].v,&e[i].w);
    }

    for(int i=0;i<m-1;i++){
        for(int j=0;j<m-i-1;j++){
            if(e[j].w > e[j+1].w){
                temp=e[j];
                e[j]=e[j+1];
                e[j+1]=temp;
            }
        }
    }

    for(int i=0;i<n;i++){
        parent[i]=i;
    }

    printf("\nMST:\n");

    for(int i=0,edges=0;i<m && edges<n-1;i++){

        int u=find(e[i].u);
        int v=find(e[i].v);

        if(u!=v){
```

```

        printf("%d %d %d\n",e[i].u,e[i].v,e[i].w);

        cost+=e[i].w;
        edges++;

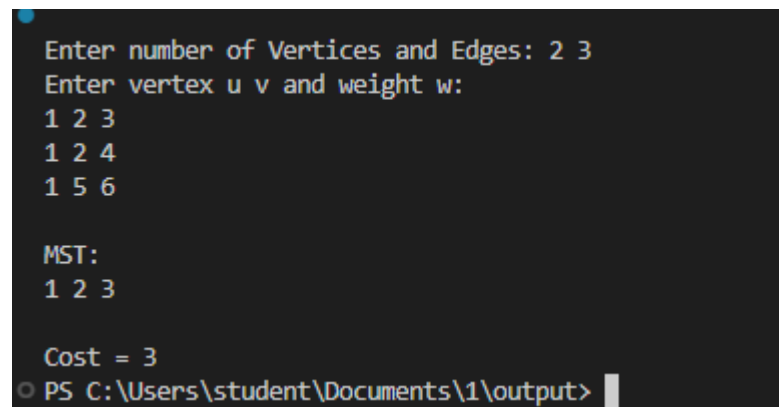
        parent[u]=v;
    }
}

printf("\nCost = %d",cost);

return 0;
}

```

OUTPUT



```

Enter number of Vertices and Edges: 2 3
Enter vertex u v and weight w:
1 2 3
1 2 4
1 5 6

MST:
1 2 3

Cost = 3
PS C:\Users\student\Documents\1\output>

```

Lab program 9:

Implement Fractional Knapsack using Greedy technique.

SOURCE CODE

```
#include <stdio.h>

int main(){
    int n,rem_weight;
    float total_profit=0;

    printf("\nEnter number of item: ");
    scanf("%d",&n);

    int obj[n],p[n],w[n];

    printf("\nEnter objects: ");
    for(int i=0;i<n;i++)
        scanf("%d",&obj[i]);

    printf("\nEnter profits: ");
    for(int i=0;i<n;i++)
        scanf("%d",&p[i]);

    printf("\nEnter weights: ");
    for(int i=0;i<n;i++)
        scanf("%d",&w[i]);

    printf("\nEnter remaining weight: ");
    scanf("%d",&rem_weight);

    float pw[n];
    for(int i=0;i<n;i++)
        pw[i]=(float)p[i]/w[i];

    for(int i=0;i<n;i++){
        for(int j=i+1;j<n;j++){
            if(pw[i]<pw[j]){
                float temp=pw[i];pw[i]=pw[j];pw[j]=temp;
                temp=obj[i];obj[i]=obj[j];obj[j]=temp;
                temp=p[i];p[i]=p[j];p[j]=temp;
                temp=w[i];w[i]=w[j];w[j]=temp;
            }
        }
    }
}
```

```

for(int k=0;k<n;k++){
    if((rem_weight-w[k]) >= 0){
        rem_weight-=w[k];
        total_profit+=p[k];
    }
    else{
        total_profit+=((float)rem_weight/w[k])*p[k];
        break;
    }
}
printf("\n\nFractional Knapsack:");
printf("\nObject\tProfit\tWeight");
for(int i=0;i<n;i++)
    printf("\n%d\t%d\t%d\t\n",obj[i],p[i],w[i]);
printf("\n\nTotal Profit is: %f\n",total_profit);
return 0;
}

```

OUTPUT

```

Enter number of item: 5

Enter objects: 1
2
3
4
5

Enter profits: 25 20 15 40 50

Enter weights: 3 2 1 4 5

Enter remaining weight: 6

Fractional Knapsack:
Object  Profit  Weight
3       15      1
2       20      2
4       40      4
5       50      5
1       25      3

Total Profit is: 65.000000
PS C:\Users\student\Documents\1\output> 

```

Leetcode 455: Assign Cookies

Problem List

455. Assign Cookies

Solved

Easy

Topics

Companies

Assume you are an awesome parent and want to give your children some cookies. But, you should give each child at most one cookie.

Each child i has a greed factor $g[i]$, which is the minimum size of a cookie that the child will be content with; and each cookie j has a size $s[j]$. If $s[j] \geq g[i]$, we can assign the cookie j to the child i , and the child i will be content. Your goal is to maximize the number of your content children and output the maximum number.

Example 1:

Input: $g = [1,2,3]$, $s = [1,1]$

Output: 1

Explanation: You have 3 children and 2 cookies. The greed factors of 3 children are 1, 2, 3. And even though you have 2 cookies, since their size is both 1, you could only make the child whose greed factor is 1 content. You need to output 1.

Example 2:

Input: $g = [1,2]$, $s = [1,2,3]$

Output: 2

Explanation: You have 2 children and 3 cookies. The greed factors of 2 children are 1, 2. You have 3 cookies and their sizes are big enough to gratify all of the children, You need to output 2.

Constraints:

5.1K 354 66 Online

Code

```
1 #include <stdlib.h>
2
3 int cmp(const void* a, const void* b) {
4     return (*(int*)a - *(int*)b);
5 }
6
7 int findContentChildren(int* g, int gSize, int* s, int sSize) {
8     qsort(g, gSize, sizeof(int), cmp);
9     qsort(s, sSize, sizeof(int), cmp);
10    int i = 0, j = 0, count = 0;
11    while (j < sSize && i < gSize) {
12        if (g[i] <= s[j]) {
13            count++;
14            i++;
15        }
16        j++;
17    }
18 }
```

Ln 19, Col 2

Testcase

Test Result

Accepted Runtime: 0 ms

Case 1 Case 2

Input

g =

[1,2,3]

s =

[1,1]

Output

1

Expected

Runtime

31 ms | Beats 9.86%

Memory

12.70 MB | Beats 38.21%

0% 5% 10%

5ms 10ms 15ms 20ms 25ms 30ms

5ms 10ms 15ms 20ms 25ms 30ms

Lab program 10:

From a given vertex in a weighted connected graph, find shortest paths to other vertices using Dijkstra's algorithm.

SOURCE CODE

```
#include <stdio.h>
#define INF 99999

int main(){

    int n,src;

    printf("Enter number of vertices: ");
    scanf("%d",&n);

    int cost[n][n],dist[n],vis[n];

    printf("Enter adjacency matrix:\n");

    for(int i=0;i<n;i++)
        for(int j=0;j<n;j++)
            scanf("%d",&cost[i][j]);

    printf("Enter source vertex: ");
    scanf("%d",&src);

    for(int i=0;i<n;i++){
        dist[i]=cost[src][i];
        vis[i]=0;
    }

    dist[src]=0;
    vis[src]=1;

    for(int i=1;i<n;i++){

        int min=INF,u=-1;

        for(int j=0;j<n;j++){
            if(!vis[j] && dist[j]<min){
                min=dist[j];
                u=j;
            }

            vis[u]=1;

            for(int v=0;v<n;v++)
                if(!vis[v] && dist[u]+cost[u][v]<dist[v])
                    dist[v]=dist[u]+cost[u][v];
```

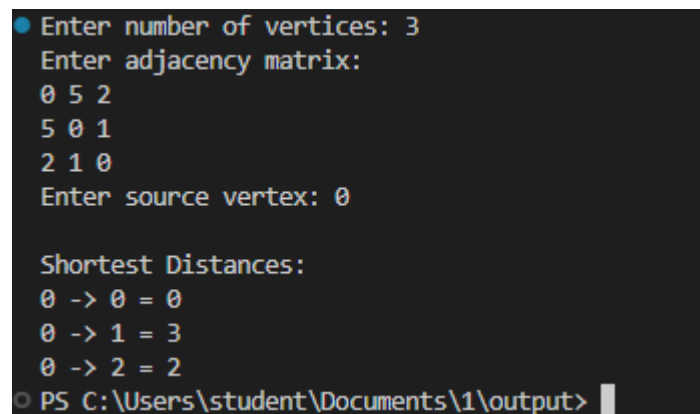
```
}

printf("\nShortest Distances:\n");

for(int i=0;i<n;i++)
    printf("%d -> %d = %d\n",src,i,dist[i]);

return 0;
}
```

OUTPUT



```
Enter number of vertices: 3
Enter adjacency matrix:
0 5 2
5 0 1
2 1 0
Enter source vertex: 0

Shortest Distances:
0 -> 0 = 0
0 -> 1 = 3
0 -> 2 = 2
PS C:\Users\student\Documents\1\output>
```

Lab program 11:

Implement “N-Queens Problem” using Backtracking.

SOURCE CODE

```
#include <stdio.h>
#include <stdlib.h>
int x[20], n;

int place(int k, int i){
    for(int j=1; j<k; j++){
        if(x[j]==i || abs(x[j]-i)==abs(j-k))
            return 0;
    }
    return 1;
}

void nqueen(int k){
    for(int i=1; i<=n; i++){
        if(place(k,i)){
            x[k]=i;
            if(k==n){
                printf("\nSolution:\n");
                for(int r=1; r<=n; r++){
                    for(int c=1; c<=n; c++){
                        if(x[r]==c)
                            printf("Q ");
                        else
                            printf(". ");
                    }
                    printf("\n");
                }
                printf("\n");
            }
            else
                nqueen(k+1);
        }
    }
}

int main(){

    printf("Enter value of n: ");
    scanf("%d",&n);

    nqueen(1);

    return 0;
}
```

OUTPUT

```
● Enter value of n: 4

Solution:
. Q . .
. . . Q
Q . . .
. . Q .

Solution:
. . Q .
Q . . .
. . . Q
. Q . .

PS C:\Users\student\Documents\1\output>
```